

Reviewer Pack — Arithmetic-Geometric Mean Invariant Bounds Tree-Like Resolut...

Entry c32c28fc2190 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 09:24:20 UTC

Arithmetic-Geometric Mean Invariant Bounds Tree-Like Resolution Proof Size

Entry ID: c32c28fc2190 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 09:24:20 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (Arithmetic-geometric mean inequality) **Field B** (complexity object): Complexity Theory: Tree-like Resolution Proof Complexity

Statement:

{‘quantitative’: ‘ $E[X(\text{instance})] = \Theta(\text{AGM}(n)^k)$ where k is a constant, for all instances of size $n \leq 40$ and any tree-like resolution proof.’, ‘specific’: ‘For any Boolean formula F with n variables and m clauses, the arithmetic-geometric mean (AGM) of the size of each clause (as an integer) raised to a power k , where k is at least $1/2$ but less than 1 , bounds the depth of the tree-like resolution proof for F . This invariant holds up to constant factors.’, ‘falsifier’: ‘A counterexample would be an instance with clauses having AGM significantly less than a power of its size that still requires a tree-like resolution proof of depth more than $\Theta(\text{AGM}(n)^k)$.’}}

Rationale (proposer’s reasoning):

{‘explanation’: ‘The arithmetic-geometric mean inequality has been used to derive lower bounds for the lengths of certain sequences, such as those in number theory and geometry. Applying this concept to complexity theory could potentially reveal new structural properties of resolution proofs.’, ‘potential’: ‘This bridge between arithmetic geometry and complexity theory may expose a deeper connection between the geometric properties of clauses and the complexity of finding resolutions.’}}

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e90deec7ff1d15d4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the average depth of the tree-like resolution proofs for all instances with $n \leq 40$ variables is within a factor of k of the AGM of clause sizes raised to

the power k , across at least 30 seeds. The conjecture is falsified if any seed produces an instance where the depth exceeds this bound by more than a factor of 2.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'arithmetic-geometric mean inequality' AND 'tree-like resolution proof complexity' - 'quantitative bounds on AGM for resolution proofs' AND 'Boolean formulas' - 'invariant depth of tree-like resolution proofs' AND 'AGM(n)^k theorem'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def agm(values):
        if not values:
            return 0
        product = 1
        sum_values = 0
        for v in values:
            product *= v
            sum_values += v
        n = len(values)
        return (product ** (1/n)) * (sum_values / n)

    def depth_of_resolution_proof(clause_sizes):
        if not clause_sizes:
            return 0
        max_size = max(clause_sizes)
        return math.ceil(math.log2(max_size))

    n = random.choice([30, 35, 40])
    m = random.randint(1, 2*n)
    clauses = [random.randint(1, n) for _ in range(m)]
```

```

clause_sizes = [len(clause) for clause in clauses]

k = random.uniform(0.5, 1)
agm_value = agm([size ** k for size in clause_sizes])
depth = depth_of_resolution_proof(clause_sizes)

return {
    "metric_name": "Depth of Resolution Proof",
    "metric_value": depth,
    "instances_tested": 1,
    "conjecture_holds": depth <= agm_value * 2 and depth >= agm_value / 2,
    "counterexample": "" if depth <= agm_value * 2 and depth >= agm_value / 2 else f"Depth {de}
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000) for _ in range(30)]

    results = []
    total_depth = 0
    total_instances = 0

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)
        total_depth += trial_result["metric_value"]
        total_instances += trial_result["instances_tested"]

    mean_depth = total_depth / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_depth) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_depth} std={std_dev} support_fraction={support_fractions}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Depth exceeds bounds for AGM\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_support_fraction support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7d0de44b.py", line 64, in <module>

```

    trial_result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7d0de44b.py", line 41, in run_trial

```

    clause_sizes = [len(clause) for clause in clauses]
    ~~~~~

```

TypeError: object of type 'int' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified according to the pre-registered support and falsification conditions. | next: Review the test code for potential bugs that cause it to crash and run the test again with a different set of seeds to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13117
2	preregistration	ollama_remote	glm4:latest	0	0	5769
3	novelty	ollama_remote	glm4:latest	0	0	4912
4	novelty	ollama_remote	glm4:latest	0	0	5481
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15971
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9343
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8012
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7794
9	judge	ollama_remote	glm4:latest	0	0	8328

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 78725 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c32c28fc2190.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c32c28fc2190.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c32c28fc2190.tar.gz` (if generated)